extreme_memory_optimizer.py

```
import gc
import os
import psutil

class MemoryOptimizer:

@staticmethod
def get_memory_usage():
"""Get current memory usage in MB"""
process = psutil.Process(os.getpid())
return process.memory_info().rss / (1024 * 1024)

@staticmethod
def optimize_memory(threshold_mb=400):
current_usage = MemoryOptimizer.get_memory_usage()
if current_usage > threshold_mb:
print(f"[Memory] WARNING: Memory usage is high ({current_usage:.1f} MB). Forcing garbage
collection.")
gc.collect()
return True
return False

@staticmethod
def limit_list(data_list, max_items=100):
if len(data_list) > max_items:
print(f"[Memory] Limiting list from {len(data_list)} to {max_items} items to save
memory")
return data_list[:max_items]
return data_list

@staticmethod
def clear_variables(*variables):
for var in variables:
var = None
gc.collect()

# Initialize memory optimization on import
gc.enable()
print(f"[Memory] Optimizer initialized. Current memory usage:
{MemoryOptimizer.get_memory_usage():.1f} MB")
```